Boston Crime Forecasting and Classification Analysis

DS 4420, Spring 2026

SioWa Luo, Li Zou, Shijie Lin
Professor Eric Gerber

April 15, 2026

Introduction

Urban crime analysis is one of the most essential application areas that is well implemented with machine learning analysis. Police departments will collect detailed records for each incident, such as information about time, location, and type of crime. These data can create an opportunity to identify patterns that are not previously seen through the descriptive stats from the ML model. In this project, we will apply three ML methods to the Boston Police Department's Crime Incident Reports dataset to study two related problems of forecasting overall monthly crime volume and predicting whether an individual incident involves a shooting.

These two problems are essential to address crime from a complementary perspective. Forecasting monthly crime volume helps the departments with resource allocation at the city level. Classifying shooting incidents can help police departments to focus on identifying high risk situations at the individual case level. By addressing those two problems, this analysis can capture both long-term trends and immediate risk factors. This structure allows the project to provide a better understanding of crime patterns in Boston while addressing specific public safety concerns.

Literature Review

Previous research shows that time series models are effective tools for analyzing crime trends over time. Studies such as Chen et al. (2008) demonstrate that ARIMA-based models can capture seasonal patterns in crime data and outperform simple baseline methods. Their research shows that first differencing is effective when they remove long-run trends, and then the ACF/PACF plots can provide a guide to select the lag order. This approach provides a strong foundation for forecasting aggregate crime levels and gives us a hint that we can use a SARIMA model for monthly crime prediction.

Neural Networks are a widely used method for crime classification analysis. Chun et al. (2019) found that deep learning models can identify complex nonlinear relationships that are spotted in crime data. Some events are very rare, but Neural Networks work well in predicting rare events with class weights. Their work highlights the importance of addressing class imbalance, which is a common challenge in crime analysis. This motivates us to use data balancing techniques when training our MLP model for handling shooting classification.

The Bayesian model offers a different perspective by focusing on probabilistic inference and uncertainty. Marchant et al. (2018) apply Bayesian regression to crime related data using Markov Chain Monte Carlo method (MCMC) with Metropolis Hasting to get the full posterior distribution over model parameters. They implement a semi-parametric spatial demographic model on New South Wales crime analysis. Their results show that Bayesian models can effectively capture uncertainty in prediction, which is important in predicting rare cases. This is relevant to our project, where a shooting incident is the rarest case in all events. We can use Bayesian logistic regression to quantify uncertainty and provide more informative predictions than other standard point estimate models.

Adding on to all the above studies, Jenga et al. (2023) provide a broader systematic review on ML methods for crime prediction. The finding shows ML methods such as neural networks, logistic regression, and tree based models can provide strength to the analysis. This finding motivates us to combine a time series model with two classification models, MLP and Bayesian, on the same dataset to strengthen our interpretation and deal with uncertainty.

Data Collection

The dataset comes from the Boston Police Department's Crime Incident Reports, available through Analyze Boston at data.boston.gov. We downloaded 2015 to 2026 data and combined them into a single dataset which contains around 906,000 records. Each row represents one reported incident and includes features such as offense code, district, data and time, day of week, and geographic coordinates. These variables allow us to analyze both when and where the crimes occur in the city of Boston.

For our data cleaning process, first, we remove the records that contain missing values in important columns such as month, hour, day of week, and district. Next, we standardize the shooting indicator to binary variables. Each incident is labeled as either involving a shooting or not. We dropped a small number of rows that had district values of external or outside of Boston to narrow our findings towards urban areas only. After the cleaning process, the dataset contains around 900,000 records across 12 districts with around 6,700 shooting incidents (around 0.7% among all records).

A key challenge we face in this dataset is class imbalance. Shooting incidents make up only a very small portion of total observations. This can lead our MLP and Bayesian model to favor the majority class. To deal with this problem, we are using an undersampling approach where we randomly select a subset of non shooting incident records so that the number of non shooting cases matches the number of shooting cases. The balanced dataset contains roughly 13,400 records. This is done to prevent both the MLP and the Bayesian model from predicting "no shooting" for every observation, which would achieve 99% accuracy but no useful classification.

After the preprocessing stage, we split the dataset into a training and testing set with an 80/20 split. The training is used to estimate model parameters, while testing is for evaluating predictive performance. This setup ensures all reported results reflect the models' ability to generalize to new data.

Methodology

Time Series (Python): The first method we implement is the SARIMA model to forecast monthly crime counts. We build the model by aggregating incident records by year and month and counting the total number of incidents each period. The results contain 129 monthly observations. Before fitting the model, we perform an ADF test to check whether the time series is stable over time. When we apply both regular differencing and seasonal differencing of d = 1 and D =1 with a 12 month cycle, the series becomes stationary with p = 0.01. Based on these results, we use the SARIMA model that includes both types of differencing. Then we create an ACF and PACF plot to understand the current value related to the past value. We test multiple combinations and run a grid search over possible values of parameters p and q from 0 to 2 and seasonal parameters P and Q 0 or 1, while we keep the two differencing parameters we did before with seasonal periods = 12. We compare their performance with the AIC score and select the lowest AIC as the best fit. The best model is SARIMA(1,1,2)(0,1,1,12) with AIC of 1299.3. We add a binary indicator for COVOID -19 period (March 2020 to December 2021) to account for structural changes in crime patterns. This adjustment enhances the model fit and reduces AIC to 1293.9. The model was evaluated on a 12-month holdout test set (February 2025 through January 2026). We also ran a rolling-origin evaluation, re-fitting on an expanding window and predicting one month at a time. This will give us a more realistic estimate of out-of-sample forecast accuracy.

MLP Neural Network (Python): The second method we implement is MLP built from scratch using NumPy. The network has one input layer, two hidden layers with 64 and 32 nodes, both using ReLU activation, and a single sigmoid output unit for binary classification. We use He initialization for the weights and train the model using mini-batch gradient descent with batch size 256 and learning rate 0.01 over 200 epochs. The input feature will capture both temporal and spatial patterns. We encode the HOUR and MONTH variables using sin/cos transformation to preserve their cyclical nature. We also include one hot encoding for DAY_OF_WEEK and DISTRICT, as well as latitude and longitude coordinates. To find the best classification threshold, we test thresholds from 0.05 to 0,95 and pick the one that gives the highest F1 score on the test set. This can help balance precision and recall, which is essential for detecting shooting events. We also evaluate three different architectures, a shallow one with 32 nodes, a medium one with 64 and 32, and a deep one with 64, 32, and 16, to confirm our chosen setup is reasonable. The medium one provides the best balance between performance and stability, and we use it as the final model.

Bayesian Logistic Regression (R): The third method we implement is a Bayesian logistic regression using the brms package in R to classify whether a crime incident involves a shooting. The target variable is a binary shooting indicator. We use MONTH, HOUR, DAY_OF_WEEK, and DISTRICT as our input features. Before fitting the model, we check the default priors provided by brms. The model is trained using 4 chains with 2000 iterations each and 500 warmup iterations. This produces a total of 6000 posterior samples after warmup draws. All Rhat values equal 1, which shows that the chain has converged and the posterior estimates are reliable. We evaluated the mode with 20% of the test set and performed a posterior predictive check. Then we draw 200 samples from the posterior distribution and compute the test accuracy for each sample. This allows us to check how stable the model predictions are across different parameters and understand uncertainty in our performance estimates.

Results and Interpretation

Time Series Results

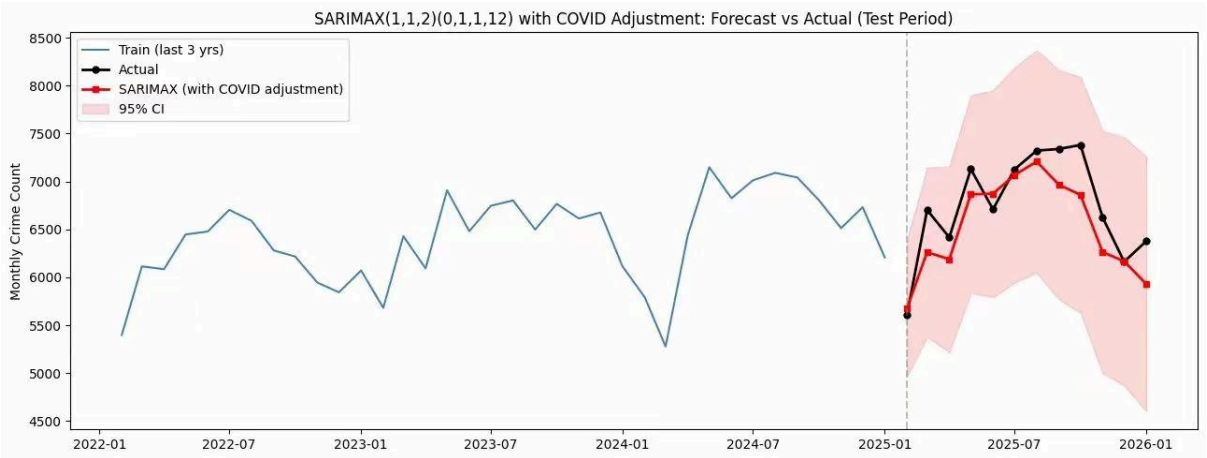
The best fitting model is SARIMA(1,1,2)(0,1,1,12) with the Covid indicator. The original model itself has an AIC score of 1299.3, and with Covid indicator the AIC score has improved to 1293.9. This finding shows that the structural break during the pandemic is statistically meaningful. The COVID coefficient is highly significant with $p = 0.0005$ and shows a 719 crime reduction per month during the pandemic (March 2020 to December 2021). These results align with our expectations because lockdowns lower the public activity that is likely to benefit the lower crime level.

The table below shows the forecast evaluation results. The rolling 1 step ahead strategy performs the best, followed by the 12 step holdout forecast, then the seasonal baseline at the bottom of performance. This aligns with our expectation because the forecast will accumulate the uncertainty level as we have longer horizons. The rolling model reduces RMSE by 38.6%, and this shows a substantial improvement in predictive accuracy.

Model	MAE	RMSE	MAPE

Naive Seasonal Baseline	320	494	4.8%
SARIMAX (12-step holdout)	254	303	3.7%
SARIMAX (rolling 1-step)	211	258	3.2%

The plot below compares our model forecast to actual crime volume over the 12 month period. The model captures the overall season pattern nicely and correctly identifies the summer peak and winter decline. But in the end, the model seems to underestimate the crime level in the fall and winter months. The actual value lies in the upper bound of the 95% CI. This shows that the model captures the general trend well, but we may not fully account on it for short term increases. The most accurate prediction is at around December 2025, where the forecast is almost in line with the actual volume, with only 0.1% error. The largest error occurred in October 2025 and January 2026 with an error of about 7%.



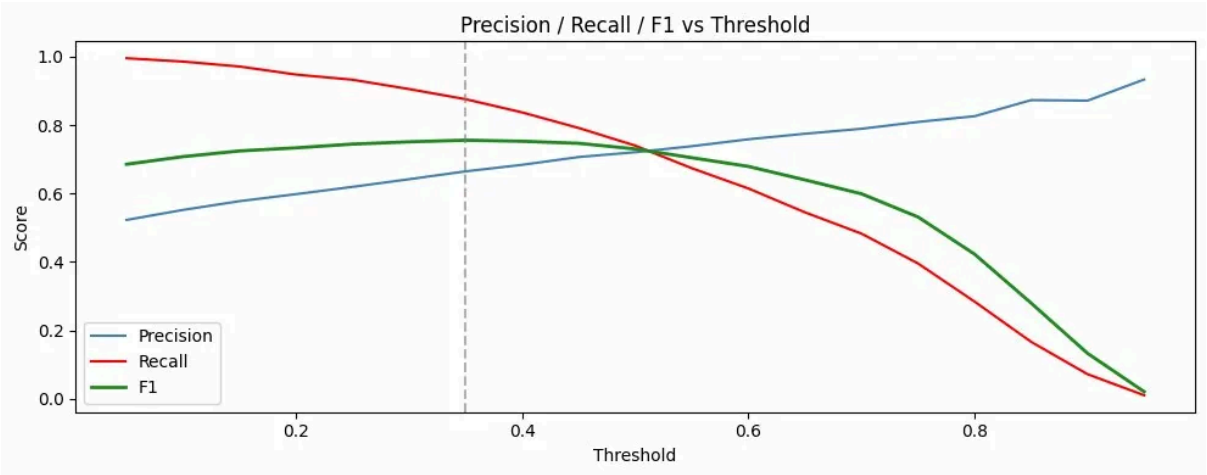
The residual analysis shows some deviations from ideal model assumptions. The Ljung-Box test says that the remaining autocorrelation at lag 12, $p = 0.007$. This suggests some seasonal structure is not fully captured. But when lag is 24 $p = 0.155$, which is fitting fine. Overall, the model performs well with a low MAPE of 3.2%, indicating that the model's forecasts are reliable for real world planning.

MLP Results

The MLP model with the medium architecture of 64 and 32 hidden units happens to achieve the best overall classification performance. At the optimum threshold of 0.35, the model has an accuracy of 71.5%, a precision of 66.5%, a recall of 87.6%, and an F1 score of 0.756. We select this threshold by evaluating values from 0.05 to 0.95 and choosing the one that maximizes the F1 score. The threshold below 0.5 is more appropriate because the model prioritizes recall as failing to identify a true shooting incident is more costly than producing a false positive. The threshold we choose (0.35) is within a reasonable range where the model identifies the majority of shooting events while having reasonable precision.

The plot below shows all three scores with the range of evaluated thresholds. Precision increases as the threshold rises while recall decreases. All curves intersect near 0.5; this means that at this threshold, the model is more balanced but less effective in classification.

The F1 score peaks at our chosen threshold of 0.35, where the model captures 87.6% of all shooting incidents while maintaining acceptable precision.

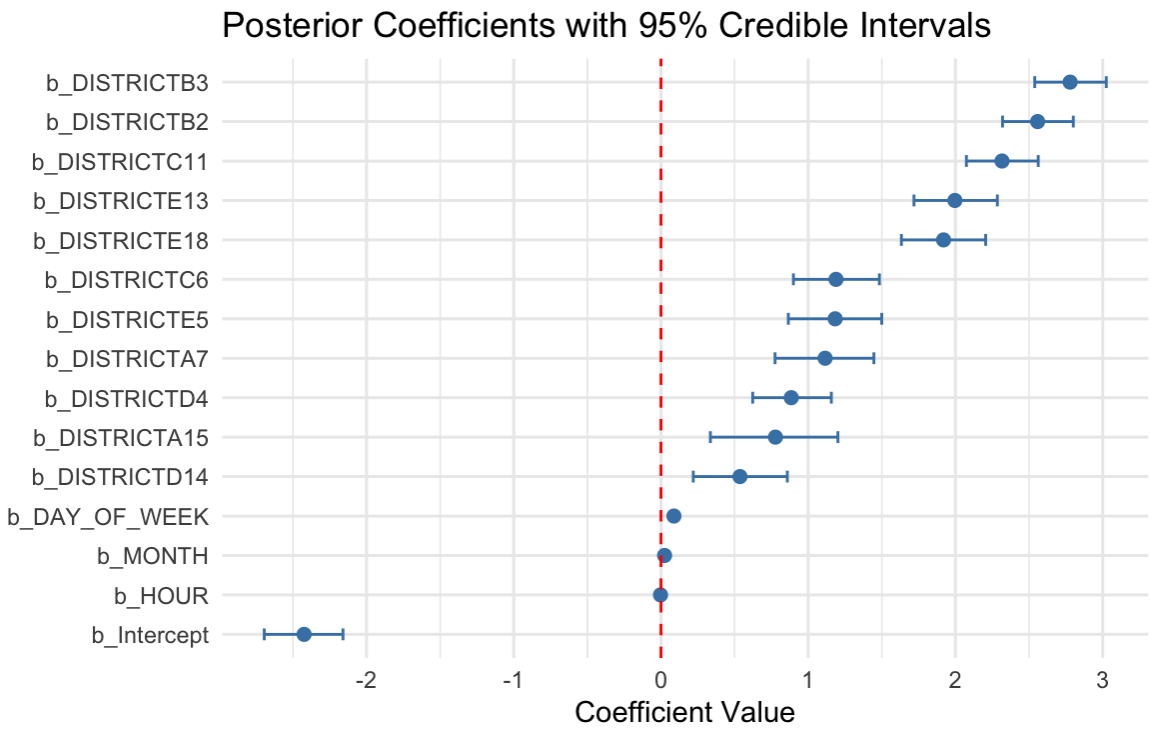


The training process shows a stable convergence. Both training and test loss decrease over 200 epochs. As a result, we get 0.519 and 0.549. The small gap suggests that the model generalizes well and it does not suffer from significant overfitting.

We also compare three different architectures with the same threshold of 0.35. The medium architecture of 64, 32 (our final pick) has the highest F1 score of 0.756, followed by the shallow model (hidden layer of 32) of 0.754, and the deep model (hidden layer 64, 32, 16) of 0.752. The differences are not significant, and this finding shows that increasing model complexity didn't always improve performance. This finding shows that the temporal and spatial features provide limited additional signal for classifying shooting incidents even with more complex networks.

Bayesian Logistic Results

For the Bayesian logistic regression, the model converges well, with all Rhat values at 1.00 and trace plots showing no obvious issues. Looking at the posterior coefficients, district location turns out to be the strongest predictor of shooting risk. Districts B2 (Roxbury), B3 (Mattapan), and C11 (Dorchester) have the largest positive coefficients with credible intervals that do not cross zero, meaning these districts consistently show higher shooting risk. All other district coefficients are also positive relative to the reference district A1 (Downtown), which suggests Downtown has the lowest shooting risk overall. DAY_OF_WEEK is significant but small in magnitude, meaning later days in the week are associated with slightly higher shooting probability. MONTH and HOUR do not show a clear effect on shooting probability because their coefficients are close to 0.



On the test set, the Bayesian model achieves an accuracy of 67.0%, precision of 0.654, recall of 0.785, and F1 of 0.714. The posterior predictive check shows test accuracy tightly distributed around 0.670, which indicates the model performs consistently across different posterior samples, suggesting our results are reliable.

Conclusion:
In this project, we apply three machine learning methods to Boston crime data to address two problems: forecasting monthly crime volume and classifying shooting incidents.

For the forecasting, our time series model achieves an MAPE of 3.7% on the 12-month holdout test and reduces RMSE by 38.6% compared to the naive seasonal baseline. The strong seasonal component confirms that Boston crime follows a clear annual cycle, and the COVID indicator successfully captures the structural break in 2020. One limitation of our SARIMA model is that the residuals still show some remaining seasonal autocorrelation and have heavier tails than expected. This means the model may not be fully capturing all the seasonal patterns in the data, and a more flexible approach like LSTM could potentially do better.

For the classification task, both the Bayesian model and the MLP produce reasonable results with F1 scores of 0.714 and 0.756. District is the dominant predictor in both models, with B2 (Roxbury) and B3 (Mattapan) consistently showing higher shooting risk. The Bayesian model gives us a distribution over each coefficient rather than just a single estimate, which makes it easier to see which features are reliable predictors. The MLP is harder to explain but uses more spatial information through the latitude and longitude features, which is likely why it catches more shooting incidents correctly.

Future Improvement:

One limitation we want to point out is that our undersampling approach throws away a large portion of the data. Shooting incidents make up only around 0.7% of all records, and models trained on balanced data may not work as well when applied to real world data where shootings are this rare.

Besides, other areas for future work include adding external features like weather or neighborhood demographic data. For the time series model, testing LSTM or other more complex models that could potentially improve forecast accuracy. For the classifiers, building separate models for each district or adding more neighborhood-level features could help capture local patterns that a single model might miss.

Appendix

Python — SARIMA Training (statsmodels)

```
def fit_sarima(train_series, order, seasonal_order, exog = None):
    """
    Fit a SARIMA (or SARIMAX with exog) model.

    Parameters:
    train_series: pd.Series of training data
    order: tuple (p, d, q) non-seasonal order
    seasonal_order: tuple (P, D, Q, s) seasonal order
    exog: optional exogenous variable array for SARIMAX
    """
    model = SARIMAX(
    train_series,
    order = order,
    seasonal_order = seasonal_order,
    exog = exog,

    # Let optimizer explore freely
    enforce_stationarity = False,
    enforce_invertibility = False
    )

    return model.fit(disp = False, maxiter = 500)
```

Python — MLP Training (NumPy, manual implementation)

```
def relu(z):
   return np.maximum(0, z)

def relu_derivative(z):
   return (z > 0).astype(float)

def sigmoid(z):
   z = np.clip(z, -500, 500)
   return 1.0 / (1.0 + np.exp(-z))

def initialize_weights(layer_dims, seed = 42):
```

```
    """He initialization: scale by sqrt(2 / fan_in), good for ReLU."""
    rng = np.random.RandomState(seed)
    params = {}

    for l in range(1, len(layer_dims)):
        scale = np.sqrt(2.0 / layer_dims[l - 1])
        params[f"W{l}"] = rng.randn(layer_dims[l - 1], layer_dims[l]) * scale
        params[f"b{l}"] = np.zeros((1, layer_dims[l]))

    return params

def forward_pass(X, params, n_layers):
    """Forward prop: ReLU hidden layers, sigmoid output. Returns predictions and cache."""
    cache = {"A0": X}

    for l in range(1, n_layers):
        Z = cache[f"A{l-1}"] @ params[f"W{l}"] + params[f"b{l}"]
        cache[f"Z{l}"] = Z
        cache[f"A{l}"] = relu(Z)

    Z_out = cache[f"A{n_layers-1}"] @ params[f"W{n_layers}"] + params[f"b{n_layers}"]
    cache[f"Z{n_layers}"] = Z_out
    cache[f"A{n_layers}"] = sigmoid(Z_out)

    return cache[f"A{n_layers}"], cache

def compute_loss(y_true, y_pred):
    """Binary cross-entropy loss."""
    eps = 1e-12
    y_pred = np.clip(y_pred, eps, 1 - eps)
    bce = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

    return np.mean(bce)

def backward_pass(y_true, params, cache, n_layers):
```

```
    """Backpropagation."""
    grads = {}
    m = y_true.shape[0]
    y_col = y_true.reshape(-1, 1)

    # Output layer
    dZ = cache[f"A{n_layers}"] - y_col
    grads[f"dW{n_layers}"] = (cache[f"A{n_layers-1}"].T @ dZ) / m
    grads[f"db{n_layers}"] = np.mean(dZ, axis = 0, keepdims = True)

    # Hidden layers
    for l in range(n_layers - 1, 0, -1):
        dA = dZ @ params[f"W{l+1}"].T
        dZ = dA * relu_derivative(cache[f"Z{l}"])
        grads[f"dW{l}"] = (cache[f"A{l-1}"].T @ dZ) / m
        grads[f"db{l}"] = np.mean(dZ, axis = 0, keepdims = True)

    return grads

def update_params(params, grads, lr, n_layers):
    """Gradient descent step."""
    for l in range(1, n_layers + 1):
        params[f"W{l}"] -= lr * grads[f"dW{l}"]
        params[f"b{l}"] -= lr * grads[f"db{l}"]

    return params


def train_mlp(X_train, y_train, X_test, y_test,
              hidden_layers = [64, 32], lr = 0.01, epochs = 200,
              batch_size = 256, seed = 42, verbose = True):
    """
    Train MLP with mini-batch gradient descent.
    Returns trained params, number of layers, and loss history.
    """
```

```
    n_features = X_train.shape[1]
    layer_dims = [n_features] + hidden_layers + [1]
    n_layers = len(layer_dims) - 1

    params = initialize_weights(layer_dims, seed = seed)
    history = {"train_loss": [], "test_loss": []}
    rng = np.random.RandomState(seed)

    for epoch in range(epochs):
        # Shuffle and mini-batch update
        perm = rng.permutation(len(y_train))
        X_s, y_s = X_train[perm], y_train[perm]

        for start in range(0, len(y_train), batch_size):
            end = min(start + batch_size, len(y_train))
            y_pred, cache = forward_pass(X_s[start:end], params, n_layers)
            grads = backward_pass(y_s[start:end], params, cache, n_layers)
            params = update_params(params, grads, lr, n_layers)

        # Track loss every 10 epochs
        if epoch % 10 == 0 or epoch == epochs - 1:
            tr_p, _ = forward_pass(X_train, params, n_layers)
            te_p, _ = forward_pass(X_test, params, n_layers)
            history["train_loss"].append(compute_loss(y_train, tr_p.flatten()))
            history["test_loss"].append(compute_loss(y_test, te_p.flatten()))

        # Print every 50 epochs
        if verbose and (epoch % 50 == 0 or epoch == epochs - 1):
            print(f"  Epoch {epoch:3d}: train={history['train_loss'][-1]:.4f}
test={history['test_loss'][-1]:.4f}")

    return params, n_layers, history
```

R — Bayesian Logistic Regression Training (brms)

```
# check default priors for our model
```

```
def_prior <- default_prior(
  SHOOTING ~ MONTH + HOUR + DAY_OF_WEEK + DISTRICT,
  data   = df_train,
  family = bernoulli("logit")
)

print(def_prior)


# fit the model
set.seed(42)
brm_fit <- brm(
  SHOOTING ~ MONTH + HOUR + DAY_OF_WEEK + DISTRICT,
  family  = bernoulli("logit"),
  data    = df_train,
  prior   = def_prior,
  chains  = 4,
  iter    = 2000,
  warmup  = 500,
  cores   = 1,
  seed    = 42
)
```

Reference

Chen, Peng, Hongyong Yuan, and Xueming Shu. 2008. "Forecasting Crime Using the ARIMA Model." In Proceedings of the 5th International Conference on Fuzzy Systems and Knowledge Discovery (FSKD), 627–30. Jinan, China. https://doi.org/10.1109/FSKD.2008.222.

Chun, Soon Ae, Venkata Avinash Paturu, Shengcheng Yuan, Rohit Pathak, Vijayalakshmi Atluri, and Nabil R. Adam. 2019. "Crime Prediction Model Using Deep Neural Networks." In Proceedings of the 20th Annual International Conference on Digital Government Research (dg.o '19), 512–14. New York: Association for Computing Machinery. https://doi.org/10.1145/3325112.3328221.

Jenga, Khadija, Cagatay Catal, and Gorkem Kar. 2023. "Machine Learning in Crime Prediction." Journal of Ambient Intelligence and Humanized Computing 14: 2887–2913. https://doi.org/10.1007/s12652-023-04530-y.

Marchant, Robert, Simi Haan, Garner Clancey, and Sally Cripps. 2018. "Applying Machine Learning to Criminology: Semi-Parametric Spatial-Demographic Bayesian Regression." Security Informatics 7 (1): 1–26. https://doi.org/10.1186/s13388-018-0030-x.